Experiment 26

Title

Write a Prolog Program for Fruit and its Color using Backtracking.

Aim

To implement a Prolog program that identifies the color of a fruit using backtracking.

Objective

To understand how Prolog uses facts and backtracking to retrieve fruits and their corresponding colors.

Algorithm

Step 1: Define facts representing fruits and their colors.

Step 2: Create a rule to match a fruit with its color.

Step 3: Enter the fruit name or color through a query.

Step 4: Prolog searches the database for matching facts.

Step 5: Using backtracking, Prolog displays all possible matching fruits and colors.

Program

```
% Fruit Database

fruit(apple, red).
fruit(banana, yellow).
fruit(grape, purple).
fruit(orange, orange).
fruit(watermelon, green).

% Rule to Match Fruit and Color

fruit_color(Fruit, Color) :-
    fruit(Fruit, Color).
```

Sample Input

?- fruit_color(Fruit, Color).

Sample Output

```
| ?- consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp26.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp26.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp26.pl compiled, 11 lines read - 826 bytes written, 7 ms

yes
| ?- fruit_color(Fruit, Color).

Color = red
Fruit = apple ? ;

Color = yellow
Fruit = banana ? ;

Color = purple
Fruit = grape ? ;

Color = orange
Fruit = orange ? ;

Color = green
Fruit = watermelon

(109 ms) yes
| ?-
```

Result

Hence, the Prolog program for **Fruit and its Color using Backtracking** was executed successfully and the correct fruit-color combinations were obtained.

Experiment 27

Title

Write a Prolog Program to implement Best First Search Algorithm.

Aim

To implement the Best First Search algorithm in Prolog to find the best path from the source node to the goal node.

Objective

To understand how Best First Search selects the most promising node based on the minimum edge cost and finds a path to the goal.

Algorithm

Step 1: Define the graph using facts with edge costs.

Step 2: Specify the source node and the goal node.

Step 3: Select the adjacent node having the lowest cost.

Step 4: Continue the search while avoiding already visited nodes.

Step 5: Display the best path from the source node to the goal node.

Program

```
% Graph with Edge Costs

edge(a,b,2).
edge(a,c,5).
edge(b,d,3).
edge(b,e,6).
edge(c,f,7).
edge(d,g,2).
edge(e,g,4).
edge(f,g,1).

% Best First Search
```

```
best_first(Start, Goal, Path) :-
    bfs(Start, Goal, [Start], RevPath),
    reverse(RevPath, Path).

bfs(Goal, Goal, Visited, Visited).

bfs(Current, Goal, Visited, Path) :-
    findall(Cost-Next,
        (edge(Current, Next, Cost),
         \+ member(Next, Visited)),
        Neighbours),
    sort(Neighbours, [_-Best|_]),
    bfs(Best, Goal, [Best|Visited], Path).
```

Sample Input

```
?- best_first(a,g,Path).
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp27.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp27.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp27.pl compiled, 25 lines read - 2534 bytes written, 8 ms

yes
| ?- best_first(a,g,Path).

Path = [a,b,d,g] ? ;

(15 ms) no
| ?-

best_first(a,g,Path).

Path = [a,b,d,g] ?

(31 ms) yes
| ?-
```

Result

Hence, the Prolog program to implement the **Best First Search Algorithm** was executed successfully and the best path was obtained.

Experiment 28

Title

Write a Prolog Program for Medical Diagnosis.

Aim

To implement a Prolog program for medical diagnosis based on the symptoms of a patient.

Objective

To understand how Prolog uses facts and rules to diagnose diseases based on the symptoms provided.

Algorithm

Step 1: Define the symptoms of different patients as facts.

Step 2: Define rules that relate symptoms to diseases.

Step 3: Enter the patient name through a query.

Step 4: Prolog searches for the patient's symptoms.

Step 5: Display the diagnosed disease if the symptoms match a disease rule.

Program

```
% Symptoms of Patients

symptom(john, fever).
symptom(john, cough).
symptom(john, headache).

symptom(anu, fever).
symptom(anu, rash).

symptom(rahul, fever).
symptom(rahul, body_ache).
symptom(rahul, cough).
```

```
% Disease Rules

disease(Patient, flu) :-
    symptom(Patient, fever),
    symptom(Patient, cough),
    symptom(Patient, headache).

disease(Patient, measles) :-
    symptom(Patient, fever),
    symptom(Patient, rash).

disease(Patient, viral_fever) :-
    symptom(Patient, fever),
    symptom(Patient, body_ache),
    symptom(Patient, cough).

% Diagnosis

diagnose(Patient) :-
    disease(Patient, Disease),
    write('Disease : '),
    write(Disease), nl.
```

Sample Input

```
?- diagnose(john).

?- diagnose(anu).
```

?- diagnose(rahul).

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp28.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp28.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp28.pl compiled, 34 lines read - 2468 bytes written, 9 ms

yes
| ?- diagnose(john).
Disease : flu

true ?

(78 ms) yes
| ?- diagnose(anu).
Disease : measles

true ?

(47 ms) yes
| ?- diagnose(rahul).
Disease : viral_fever

true ?

(78 ms) yes
| ?-
```

Result

Hence, the Prolog program for **Medical Diagnosis** was executed successfully and the correct disease was diagnosed.

Experiment 29

Title

Write a Prolog Program for Forward Chaining. Incorporate Required Queries.

Aim

To implement Forward Chaining in Prolog to infer new facts from the given knowledge base.

Objective

To understand how Prolog derives new facts by applying rules to the available facts in the knowledge base.

Algorithm

Step 1: Define the initial facts in the knowledge base.

Step 2: Define inference rules to derive new facts.

Step 3: Execute the forward chaining rule.

Step 4: Prolog applies the rules to the available facts.

Step 5: Display the inferred result.

Program

```
% Facts

bird(eagle).
bird(penguin).

has_wings(eagle).
has_wings(penguin).

can_fly(eagle).

cannot_fly(penguin).
```

```
% Forward Chaining Rules

animal(X) :-
    bird(X).

flying_bird(X) :-
    bird(X),
    has_wings(X),
    can_fly(X).

non_flying_bird(X) :-
    bird(X),
    cannot_fly(X).

% Inference Rule

infer(X) :-
    flying_bird(X).

infer(X) :-
    non_flying_bird(X).
```

Sample Input

```
?- infer(X).
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl compiled, 32 lines read - 1748 bytes written, 8 ms

(16 ms) yes
| ?- infer(X).

X = eagle ? ;

X = penguin

(15 ms) yes
| ?-
```

Result

Hence, the Prolog program for **Forward Chaining** was executed successfully and the required facts were inferred.

Experiment 30

Title

Write a Prolog Program for Backward Chaining. Incorporate Required Queries.

Aim

To implement Backward Chaining in Prolog to determine whether a goal can be satisfied using the available facts and rules.

Objective

To understand how Prolog uses backward chaining to prove a goal by matching it with rules and facts in the knowledge base.

Algorithm

Step 1: Define the facts in the knowledge base.

Step 2: Define rules that relate the facts.

Step 3: Enter the goal through a query.

Step 4: Prolog searches the rules to satisfy the goal.

Step 5: Display the inferred result if the goal is proved.

Program

```
% Facts

bird(eagle).
bird(penguin).

has_wings(eagle).
has_wings(penguin).

can_fly(eagle).
cannot_fly(penguin).
```

```
% Backward Chaining Rules

flying_bird(X) :-
    bird(X),
    has_wings(X),
    can_fly(X).

non_flying_bird(X) :-
    bird(X),
    cannot_fly(X).

% Goal

goal(X) :-
    flying_bird(X).

goal(X) :-
    non_flying_bird(X).
```

Sample Input

```
?- goal(eagle).

?- goal(penguin).

?- goal(crow).
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl compiled, 28 lines read - 1628 bytes written, 9 ms
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:3: redefining procedure bird/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:3: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:6: redefining procedure has_wings/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:6: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:9: redefining procedure can_fly/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:9: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:10: redefining procedure cannot_fly/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:11: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:14: redefining procedure flying_bird/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:18: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:19: redefining procedure non_flying_bird/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:23: previous definition

(63 ms) yes
| ?- goal(eagle).

true ?

(47 ms) yes
| ?- goal(penguin).

yes
| ?-
goal(crow).

no
| ?-
```

Result

Hence, the Prolog program for **Backward Chaining** was executed successfully and the required goal was proved.